

Introspector

"Introspection is all you need."

Covenants on Bitcoin without softfork on Arkade

bonomat

BBBBD - Byron Bay BushBash BitDevs 2026

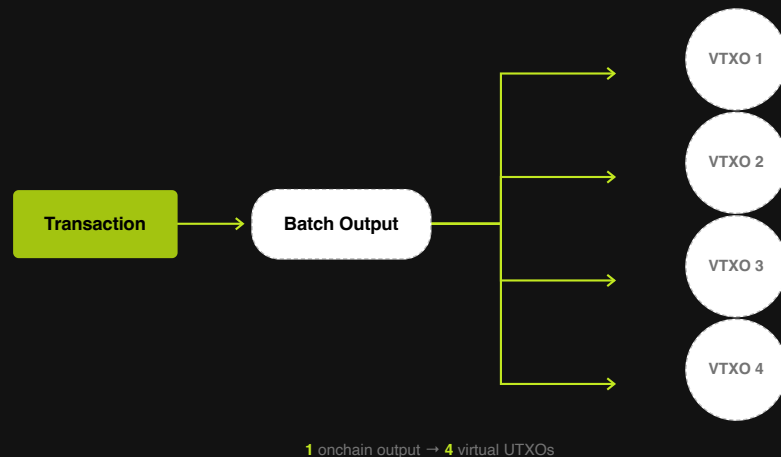
CHAPTER 01 / 05

Arkade

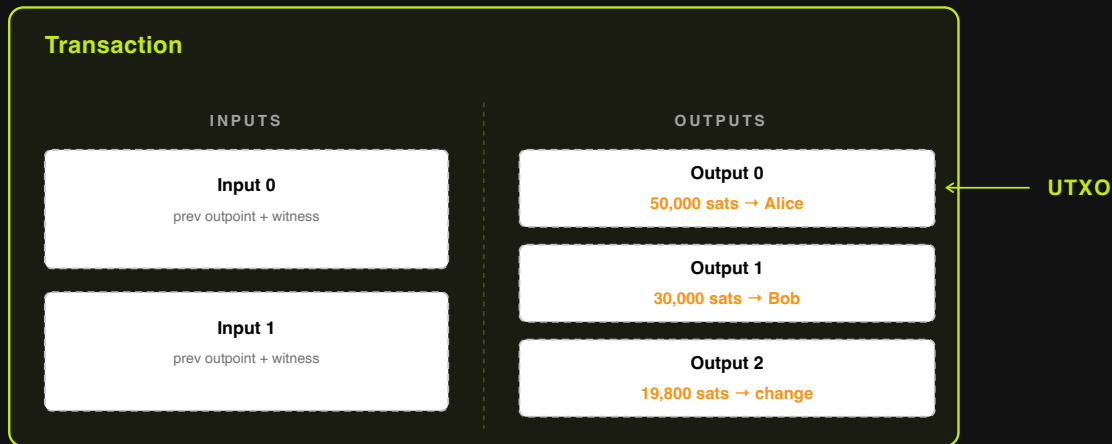
A Bitcoin L2

What is Arkade?

- A **Bitcoin L2** for fast, cheap, off-chain payments
 - anchored to a single on-chain output.
- **VTXOs** = *virtual* UTXOs that live inside that shared batch output.
- An operator (`arkd`) batches many users into one onchain tx.
- Every VTXO is a **2-of-2** between user and operator + 1 one unilateral exit path.
- Trade-off: instant + non-custodial, but **every move needs the operator to co-sign.**



Anatomy of a Bitcoin transaction



Each output has a **value** and a **scriptPubKey**. Each input references a previous output's outpoint.

A **VTXO** is one of these outputs — with a script that says *what's allowed to spend it*.

Anatomy of a VTXO

```
vtxoScript = Taproot(  
    internalKey  
:  
UNSPENDABLE,  
    scriptPaths  
:  
[  
    // Collaborative path (default)  
    checkSig(userPK) && checkSig(operatorPK),  
  
    // Unilateral exit path  
    checkSig(userPK) && relativeTimelock(exitDelay)  
]  
)
```

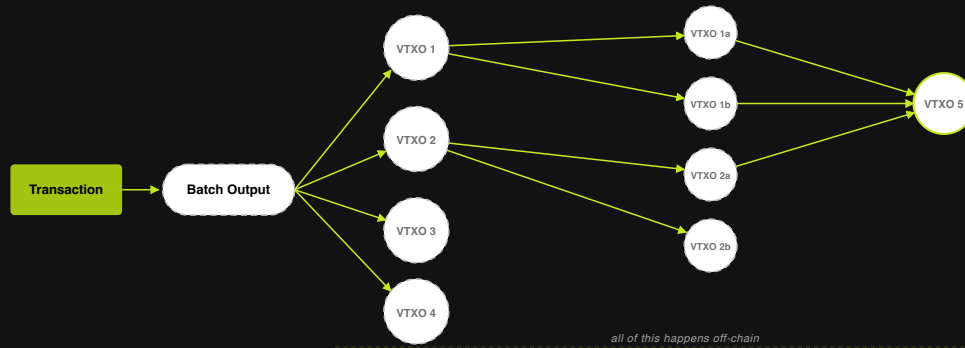
Collaborative path — *fast, off-chain*

User **and** operator co-sign every move. Instant settlement, no on-chain footprint.

Unilateral exit — *the safety valve*

After `exitDelay`, the user alone can claim the VTXO on-chain. The operator can **never** trap funds.

Spends compose, off-chain



One on-chain anchor. Many off-chain spends. **The DAG grows without touching the chain.**

What can you build on Arkade?

Instant Bitcoin payments

Lightning-grade speed — no per-user channel to open, no inbound liquidity to source. One VTXO, many spends.

Other asset and stablecoins

Issued assets ride inside VTXOs - same unilateral-exit guarantees as native BTC.

Trading & atomic swaps

Atomic swaps between Arkade assets, or cross-chain to Lightning / mainchain.

Arkade Script — advanced scripting

Extra opcodes on top of Bitcoin Script: introspection, hashing, arithmetic. Vaults, **covenants**.

CHAPTER 02 / 05

Covenants

The constraint Bitcoin Script can't express — yet

What is a covenant?

- Bitcoin Script today gates **who** can spend a UTXO.
- A covenant gates **how** it can be spent.

Regular UTXO

"anyone with key x can move this *anywhere*"

Covenant UTXO

"this can only be spent to output y , for amount z , after time T "

A spending constraint that depends on the *transaction itself* — not just signatures and preimages.

Why you'd want them

- **Lightning async receive** — pay-on-preimage without both parties online at swap time.
- **Non-interactive atomic swaps** — trade assets without online requirement.
- **Delegation** — let someone *move* your coins, but only under script-enforced rules.
- **Vaults** — funds can only move to your cold address, or anywhere else after a delay.

The common thread: each one wants the script to **see the spending transaction** — its outputs, its amounts, its shape.

What Bitcoin Script can — and can't — do

Today, Script constrains:

- *who* signs — `OP_CHECKSIG` , multisig, MuSig2
- *when* it spends — `OP_CLTV` , `OP_CSV`
- *what secret* is revealed — hashlocks

Script cannot:

- read the spending tx's outputs
- enforce "this output must look like *that*"
- check amounts, scriptPubKeys, or versions
- chain commitments across transactions

Bitcoin scripts today

The soft-fork road 🐢

OP_CTV · OP_CAT · OP_CSFS · OP_VAULT ·
SIGHASH_ANYPREVOUT

Each one is years of debate, safety review, and politics. We've been "almost there" for a while now.

Meanwhile, the pragmatic path 🐇

Run a signer that **only co-signs if your script says it's OK.**

Same expressive power. Ships today. No fork required.

That signer is the **Introspector** →

CHAPTER 03 / 05

The Introspector

A signer that only co-signs if your script says it's OK

The trick: a script-committed pubkey

The Introspector is a daemon holding a key P .

It will co-sign **any** VTXO spend — *but only if* the input commits to a script, and that script **evaluates true** against the spending tx.

Commit the script into the pubkey itself:

```
P_committed = P_introspector + H_tag("ArkScriptHash", arkadeScript) · G
```

- **BIP-341-style taproot tweaking**, with a project-specific tag.
- The script bytes never go on-chain in the path itself.
- They're encoded in an OP_RETURN.
- The Introspector **recomputes** the tweak from the revealed script.
- If the script evaluates true against the tx, it signs.
- Otherwise: silence. The coin can't move that way.

VTXO, with an Introspector leaf

```
vtxoScript = Taproot(  
  internalKey: UNSPENDABLE,  
  scriptPaths: [  
    // Covenant path — the script does the work  
    checkSig(operatorPK) && checkSig(P_committed),  
    //                                     ↑ tweaked by H_tag("ArkScriptHash", arkadeScript)  
  
    // Unilateral exit — the safety valve  
    checkSig(userPK) && relativeTimelock(exitDelay)  
  ]  
)
```

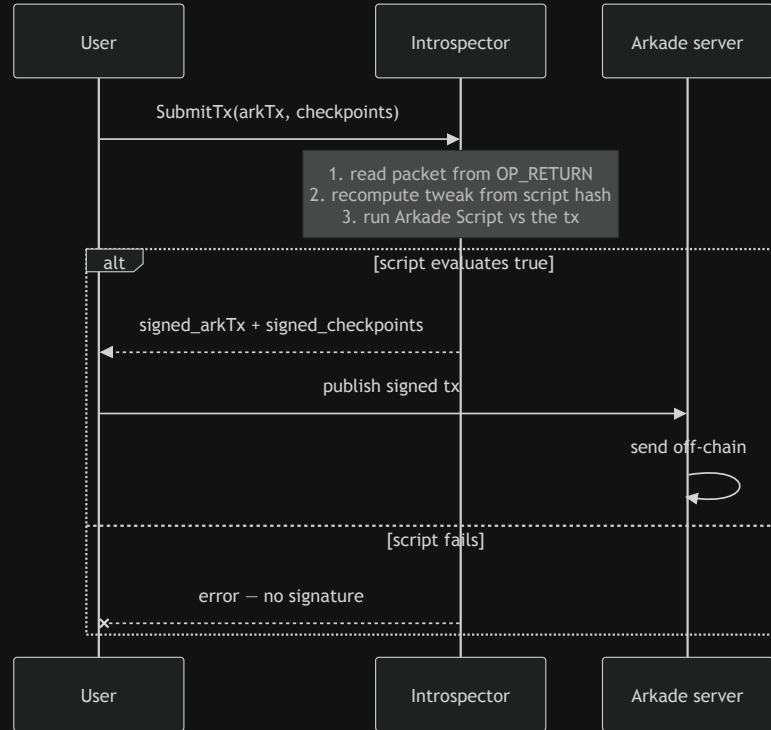
Covenant path — *async, script-enforced*

Operator + Introspector. The Introspector signs **only** if the committed Arkade script evaluates true against the spending tx. **No user signature required** — that's what makes it covenant-like.

Unilateral exit — *the safety valve*

User alone, after `exitDelay`. Operator and Introspector can both vanish; the user still walks away on-chain.

How the Introspector signs



Example: a vault

Funds can leave **instantly** to your **cold** address — or anywhere else, but **only** after a **delay**.

```
# Arkade Script – vault
# witness: <to_cold?>

OP_IF
  # fast path – instant, but ONLY to the cold address
  0   OP_INSPECTOUTPUTSCRIPTPUBKEY  # output 0: [program, version]
  <cold_program>   OP_EQUALVERIFY    # must pay cold
  <segwit_v1>      OP_EQUALVERIFY
OP_ELSE
  # spend path – anywhere, but only after the delay
  <delay>   OP_CHECKSEQUENCEVERIFY  OP_DROP
OP_ENDIF
1
```

The rule — fast path: output 0 **must** pay the committed cold address. Slow path: spend anywhere, but only once `delay` blocks have passed.

Why it's safe — a thief with your hot key still can't move funds for `delay`. You see it coming and sweep everything to cold — instantly.

CHAPTER 04 / 05

Show me the code

Real-world covenants, today

Banco: non-interactive swaps

`github.com/arkade-os/banco`

Maker creates an offer, locks funds in a covenant VTXO, then **goes offline**. **Taker** discovers the offer later and fulfills it — atomically, in one Ark tx.

Why the maker can disappear

The covenant script enforces, on the fulfillment tx:

- output 0 pays the **maker's** scriptPubKey
- output 0 value $\geq$ wantAmount (sats or assets)

No maker signature at fulfillment.

Three taptree leaves

- **Fulfill** — covenant + Introspector co-sign
- **Cancel** — maker reclaims after CLTV
- **Exit** — unilateral CSV fallback

Live walkthrough: `src/maker.ts` · `src/taker.ts` · `src/offer.ts`

CHAPTER 05 / 05

Summary